

Ejercicio 1. Método signo

Se proporciona el siguiente método

```
public static int signo(int x) {  
    if (x > 0) {  
        return 1;  
    } else if (x < 0) {  
        return -1;  
    }  
    return 0;  
}
```

Análisis:

- Qué hace:
Devuelve:
 - 1 si el número es mayor que 0
 - -1 si es menor que 0
 - 0 si es cero
- Entradas: cualquier número entero (int)
- Salidas: -1, 0, 1
- Casos:
 - Normal: positivo o negativo
 - Límite: 0

Complejidad ciclomática

Necesitas mínimo 3 tests, uno para el if, otro para else if, y otro para igual que 0

ID	Descripción	Entrada	Esperado	Estado
TC01	Nº positivo	3	1	Okey
TC02	Nº negativo	-2	-1	Okey
TC03	Nº igual a 0	0	0	Okey

Ejercicio 2. Método clasificarEdad

Se proporciona el siguiente método:

```
public static String clasificarEdad(int edad) {  
    if (edad < 0) {  
        throw new IllegalArgumentException("Edad no válida");  
    }  
  
    if (edad < 6) {  
        return "Infancia";  
    } else if (edad < 12) {  
        return "Niñez";  
    } else if (edad < 18) {  
        return "Adolescencia";  
    }  
  
    } else if (edad < 25) {  
        return "Juventud";  
    } else if (edad < 60) {  
        return "Adulthood";  
    } else {  
        return "Vejez";  
    }  
}
```

1.Complejidad ciclomática

Necesitas minimo 7 intentos,

2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.

ID	Descripción	Entrada	Esperado	Estado
TC01	Menor que 0	-2	Edad no valida	Okey
TC02	entre 0 y 6	4	Infancia	Okey
TC03	entre 6 y 12	10	Niñez	Okey
TC04	Entre 12 y 18	14	Adolescencia	Okey
TC05	entre 18 y 25	20	Juventud	Okey
TC06	entre 25 y 60	35	Adulthood	Okey
TC07	mayor que 60	70	Vejez	Okey
TC08	Igual que 6	6	Niñez	Okey
TC09	igual que 12	12	Adolescencia	Okey

TC10	igual que 18	18	Juventud	Okey
TC11	igual que 25	25	Adultez	Okey
TC12	igual que 60	60	Vejez	Okey

3. Añade además casos límite especialmente relevantes.

4. Implementa ClasificarEdadTest

Ejercicio 3. Método contarPositivos

Se proporciona el siguiente método:

```
public static int contarPositivos(int[] datos) {
    int contador = 0;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] > 0) {
            contador++;
        }
    }
    return contador;
}
```

1. Calcula la complejidad ciclomática.

1;
for 2;
if 3;

2. Diseña pruebas para cubrir: • array vacío, • array con todos negativos, • array con mezcla de positivos, negativos y ceros.

3. Justifica qué ramas cubre cada prueba.

ID	Descripción	Entrada	Esperado	Estado
TC01	Array vacío	{}	0	okey
TC02	Array con todos negativos	{-1, -5, -10}	0	okey
TC03	Array con mezcla de positivos, negativos y	{2, -3, 0, 5}	2	okey

	ceros			
--	-------	--	--	--

4. Implementa ContarPositivosTest.

Ejercicio 4. Método calificacion

```
public static String calificacion(int nota) {
    if (nota < 0 || nota > 10) {
        throw new IllegalArgumentException("Nota no válida");
    }

    return switch (nota) {
        case 0, 1, 2, 3, 4 -> "SUSPENSO";
        case 5 -> "SUFICIENTE";
        case 6 -> "BIEN";
        case 7, 8 -> "NOTABLE";
        default -> "SOBRESALIENTE";
    };
}
```

1. Calcula la complejidad ciclomática

Tiene mínimo 6 test, los 5 del switch y 1 del if

2. Diseña los casos mínimos para cubrir todas las alternativas del switch y la validación inicial.

ID	Descripción	Entrada	Esperado
TC01	Nota invalida (menor que 0)	-2	IllegalArgumentException
TC02	Suspenso(0-4)	3	SUSPENSO
TC03	Suficiente(5)	5	SUFICIENTE
TC04	Bien (6)	6	BIEN
TC05	Notable(7-8)	7	NOTABLE
TC06	Sobresaliente(9-10)	9	SOBRESALIENTE
TC07	Nota invalida (mayor que 10)	11	IllegalArgumentException

3. Implementa CalificacionTest.

4. Indica qué casos podrían agruparse con @ParameterizedTest.

3= SUSPENSO

5= SUFICIENTE

6= BIEN

7=NOTABLE

9= SOBRESALIENTE

Ejercicio 5. Crear un método esBisiesto

Implementa un método con la siguiente firma:

```
public static boolean esBisiesto(int anyo)
```

1. Implementa el método.
2. Calcula su complejidad ciclomática a partir de tu código.

Complejidad ciclomática = 4

1

```
anyo % 4 == 0
```

```
anyo % 100 != 0
```

```
anyo % 400 == 0
```

3. Diseña los casos de prueba necesarios para cubrir todas las decisiones.

ID	Descripción	Entrada	Esperado
TC01	Divisible por 4 y no por 100	2024	true
TC02	Divisible por 100 pero no por 400	1900	false
TC03	Divisible por 400	2000	true
TC04	No divisible por 4	2023	false

4. Implementa EsBisiestoTest.

Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

```
public static String jugar(String jugador1, String jugador2)
```

1. Implementar el método.
2. Calcular la complejidad ciclomática.

Complejidad ciclomática = 7

```
if (!esValida(jugador1) || !esValida(jugador2))
if (jugador1.equals(jugador2))
```

```

        if ((jugador1.equals("PIEDRA") &&
jugador2.equals("TIJERA")) ||
        (jugador1.equals("TIJERA") && jugador2.equals("PAPEL")) ||
        (jugador1.equals("PAPEL") && jugador2.equals("PIEDRA")))

```

3. Diseñar los tests mínimos para cubrir todas las ramas.

ID	Descripción	Entrada	Esperado
TC01	jugadaInvalida	"PIEDRA", "FUEGO"	IllegalArgumentException
TC02	empate	"PIEDRA", "PIEDRA"	EMPATE
TC03	piedraGanaATijera	"PIEDRA", "TIJERA"	JUGADOR1
TC04	tijeraGanaAPapel	"TIJERA", "PAPEL"	JUGADOR1
TC05	papelGanaAPiedra	"PAPEL", "PIEDRA"	JUGADOR1
TC06	ganaJugador2	"PIEDRA", "PAPEL"	JUGADOR2

4. Implementar PiedraPapelTijeraTest.

Ejercicio 7. Implementar Validador de contraseñas

Implementa el método:

```

public static int evaluarPassword(String password)

```

2. Calcula su complejidad ciclomática a partir de tu código.

Complejidad ciclomática = 6

5 if + 1

3. Diseñar los tests mínimos para cubrir todas las ramas.

ID	Descripción	Entrada	Esperado
----	-------------	---------	----------

TC01	solo minusculas	abc	1
TC02	8+ caracteres	abcdefgh	2
TC03	+mayusculas	Abcdefgh	3
TC04	+numeros	Abcdefg1	4
TC05	todos los criterios	Abcdef1!	5
TC06	sin ningún criterio	123	0